

调度序驱动的驻留构成优化：面向 NPU 的溢出代价感知方法

补充材料

高成志 黄骏 叶勤

contact@vennai.org hj992881627@outlook.com yq020319@163.com

目录

1 问题定义补充	2
1.1 DAG 与缓冲生命周期	2
1.2 三个评估视角	2
1.3 容量、地址与 spill 规则	3
1.4 Benchmark 数据描述	3
2 形式化论证与适用边界	3
2.1 Clean/dirty 代价非对称	4
2.2 近最优集合中的工作集证书	4
2.3 Belady 风格驱逐的稳定条件	5
2.4 溢出面积代理的条件性	6
2.5 Portfolio 选择的单调性	6
3 方法细节与端到端对照	7
3.1 候选序设计与溢出分离原则	7
3.2 P1: 峰值驻留与目标错配	8
3.3 P2: 调度序塑形 clean/dirty 构成	8
3.4 P3: 预取窗口与计算重叠	9
3.5 端到端参考对照	9
3.6 差距来源分析	9
3.7 运行效率	10
4 附加实验证据	11
4.1 调度序的作用	11
4.2 合成分布与适用域	13
4.3 CP-SAT oracle	14
4.4 代理目标与诊断工具	14
4.5 Clean/dirty 受控消融	16

补充材料导读

本补充材料围绕正文的三个贡献展开。§1 给出问题定义的完整形式化，固定 DAG 输入、容量参数与三阶段评估口径。§2 给出正文三个形式化结论的完整证明，包括溢出不可避免性证书（定理 1）、Belady-margin 稳定性（命题 1）和溢出面积近似保证（定理 2），并讨论各自的适用边界。§3 展开正文方法节压缩的设计细节，包括候选序生成、地址分配流程与预取窗口选择，并附与外部参考解的端到端对照。§4 提供调度序与驱逐策略的边际效应分离、合成分布上的适用域验证、CP-SAT oracle 最优性核验、以及 clean/dirty 受控消融等附加实验证据。

1 问题定义补充

正文 §3 以精简的定义建立了 P1/P2/P3 的评估框架。本节补充实验复现所需的完整形式化约定，包括 DAG 字段、评价指标的公式细节以及 spill 机制的精确参数。

1.1 DAG 与缓冲生命周期

正文仅给出节点两分类的概要，此处列出完整的字段定义。输入是一个有向无环图 $G = (V, E)$ 。节点分为两类：

1. 操作节点携带 (Id, Op, Pipe, Cycles, BuFs)，表示在某条硬件管线上的计算或数据搬运指令；BuFs 给出该指令读写的逻辑缓冲。
2. 缓存管理节点携带 (Id, Op, BuFId, Size, Type)，其中 $Op \in \{\text{ALLOC}, \text{FREE}\}$ 。每个逻辑缓冲 b 有唯一分配节点 a_b 、唯一释放节点 f_b 、大小 s_b 与所属 cache 类型 $\tau(b)$ 。

边表示必须满足的执行依赖。忽略缓存管理节点后，样例图通常从 COPY-IN 进入，经片上计算或搬运，最终由 COPY-OUT 写回片外；但本文算法不依赖具体算子族或 motif 名称。调度 $S = (v_1, \dots, v_N)$ 是覆盖所有必需节点的一条拓扑序。

1.2 三个评估视角

正文将 P1/P2/P3 浓缩为一段文字，此处展开其数学定义。三个子问题在同一图上逐步增加约束。令节点 v 对缓冲驻留量的增量为

$$M_v = \begin{cases} +s_b, & v = a_b \text{ (ALLOC)}, \\ -s_b, & v = f_b \text{ (FREE)}, \\ 0, & \text{其他节点}. \end{cases} \quad (1)$$

P1：峰值驻留。P1 只评判节点序，不要求物理地址或 spill 列表。以前缀和定义

$$V_{\text{stay}}(0) = 0, \quad V_{\text{stay}}(i) = V_{\text{stay}}(i-1) + M_{v_i}, \quad V_{\text{stay}}^{\max} = \max_{0 \leq i \leq N} V_{\text{stay}}(i). \quad (2)$$

主目标为最小化 $\max V_{\text{stay}}$ 。本文在报告中同时给出逐 cache 峰值，便于解释 P1 序与 P2/P3 之间的错配。

P2：额外 DDR 搬运量。P2 在容量约束下给出物理地址和 spill 列表，目标为最小化 spill 引入的额外数据搬运

$$\text{extra}(S) = \sum_{b \in \text{Spilled}(S)} e_b, \quad e_b = \kappa_b s_b, \quad \kappa_b = \begin{cases} 1, & b \text{ 为 clean}, \\ 2, & b \text{ 为 dirty}. \end{cases} \quad (3)$$

P3：流水执行时间。P3 在插入 spill 节点和地址复用依赖后，按多管线最早发射模型计算 makespan。记起始时间 $\sigma(v)$ 、结束时间 $\epsilon(v)$ 、增强依赖集合为 E' ：

$$\sigma(v) \geq 0, \quad \sigma(v) \geq \epsilon(u) \quad \forall (u, v) \in E', \quad \epsilon(v) = \sigma(v) + \text{Cycles}(v), \quad T = \max_v \epsilon(v). \quad (4)$$

缓存管理节点周期为 0；同一 Pipe 上的节点按调度序串行执行，不允许在管线内部重新排序。

词典键。 本文以与正文一致的词典键进行比较：P1 使用 $(\max \text{live}_{L1}, \max \text{live}_{UB}, L0 \text{ count}, T)$ ；P2 使用 $(\text{extra}, \#spills, T)$ ；P3 使用 $(T, \text{extra}, \#spills)$ 。基准定义未给出将运行时间与额外搬运合并的单一标量，因此 P3 的次级目标在本文中显式采用该词典序口径。

1.3 容量、地址与 spill 规则

正文简述了 clean/dirty 的非对称代价；此处给出地址分配与 spill 机制的完整规则。P2/P3 中，缓冲 b 以偏移 o_b 占据同一 cache 内的连续区间 $[o_b, o_b + s_b)$ 。任一时刻同时驻留且属于同一 cache 的缓冲区间不得相交，并且必须满足 $0 \leq o_b, o_b + s_b \leq \text{Cap}_{\tau(b)}$ 。容量参数见表 S1。

表 S1: 硬件 cache 类型与容量（抽象单位）。

Cache 类型	L1	UB	L0A	L0B	L0C
容量 Cap	4096	1024	256	256	512

本文称缓冲 b 为 clean，当且仅当它出现在某个 COPY-IN 节点的 BuFs 中；此时片外已有备份，换出无需写回。其余缓冲称为 dirty。相应的 spill 周期模型为

$$\text{Cycles}(\text{SPILL_IN}_b) = 2s_b + 150, \quad \text{Cycles}(\text{SPILL_OUT}_b) = \begin{cases} 0, & b \text{ 为 clean,} \\ 2s_b + 150, & b \text{ 为 dirty.} \end{cases} \quad (5)$$

若原图有 N 个节点，第 k 次 spill 生成一对隐式节点：

$$\text{Id}(\text{SPILL_OUT}_k) = N + 2(k - 1), \quad \text{Id}(\text{SPILL_IN}_k) = N + 2(k - 1) + 1. \quad (6)$$

SPILL-OUT 使用 MTE3，SPILL-IN 使用 MTE2。目标缓冲 b 的基本依赖链为

$$a_b \rightarrow \text{SPILL_OUT} \rightarrow \text{SPILL_IN} \rightarrow f_b. \quad (7)$$

已经使用过 b 的操作节点必须位于 SPILL-OUT 之前，尚未使用 b 的操作节点必须位于 SPILL-IN 之后。物理地址可在 SPILL-IN 时改变，因此可行性按驻留段检查，而不是只按逻辑生命期 $a_b \rightarrow f_b$ 检查。

地址复用会引入额外依赖：若缓冲 b 复用更早缓冲 a 的物理区间，则计时前加入 $f_a \rightarrow a_b$ 。该依赖不改变 spill 流量，却可能改变 P3 makespan。

1.4 Benchmark 数据描述

本文使用的六个公开 NPU 核内调度实例规模见表 S2。所有实验均从 DAG 图结构出发，不使用针对 Conv、FlashAttention 或 Matmul 的专用规则。

表 S2: 公开 NPU 核内调度实例规模。

任务	节点数	边数	缓冲数	操作节点数
Matmul_Case0	4160	7104	1216	1728
Matmul_Case1	30976	55040	8960	13056
FlashAttention_Case0	1716	2712	572	572
FlashAttention_Case1	6952	11184	2328	2296
Conv_Case0	2580	3869	831	918
Conv_Case1	36086	85653	12013	12060

2 形式化论证与适用边界

正文方法节以三个形式化结论为溢出代价感知的活跃度塑形建立理论基础：定理 1（溢出不可避免性证书）、命题 1（Belady-margin 稳定性）和定理 2（溢出面积近似）。本节给出它们的完整证明，并严格标注每个结论的适用条件，以避免过度声称。

三个结论从不同角度划定方法的作用域。溢出不可避免性证书以近最优调度集合上的工作集下界判定容量受限：当证书成立时，优化目标从消除溢出转为降低溢出代价。Belady-margin 稳定性解释了主要优化自由度为何是调度序而非驱逐策略：在距离主导条件下，不同 Belady 风格驱逐规则选出相同的 victim 集合，差异因此只来自序。溢出面积近似定理则为代理目标 Φ 在容量受限区域跟踪溢出流量的有效性提供条件性保证。以下各节依次给出这些结论的完整证明，并将“严格成立的定理”“需要条件的命题”和“经验观察”分开表述。

2.1 Clean/dirty 代价非对称

正文 §3 将缓冲区分为 clean 与 dirty 两类，指出二者占用相同片上空间但溢出代价相差 $2\times$ 。以下观察为这一非对称结构提供定量基础，是后续所有代价感知决策的出发点。

观察（clean/dirty 代价非对称）。 设 x 与 y 是大小均为 s 的两个缓冲， x 为 clean， y 为 dirty。若一次 spill 将 y 替换为 x ，则该次 P2 extra 从 $2s$ 降为 s ，恰好减半。

代价比的推导。 这一非对称直接来自评测指标的定义：由式 (3)，clean 缓冲的 extra 为 s ，dirty 缓冲的 extra 为 $2s$ 。因此二者之比为 $1/2$ 。该结论是评测指标本身的直接推论，不依赖特定调度器或数据集。合成消融实验中，在相同 DAG、相同 spill 数和相同峰值下，clean 储备与 dirty 储备的 extra 分别为 1536 与 3072，用于验证实现是否忠实反映该定义。

2.2 近最优集合中的工作集证书

给定拓扑序 S ，记节点 v 在序中的位置为 $\text{pos}_S(v)$ 。缓冲 b 的逻辑生命期为

$$I_S(b) = [\text{pos}_S(a_b), \text{pos}_S(f_b)). \quad (8)$$

cache c 在位置 t 的逻辑 live 工作集为

$$W_c^S(t) = \sum_{b: \tau(b)=c, t \in I_S(b)} s_b, \quad W_c^S = \max_t W_c^S(t). \quad (9)$$

对候选序集合 $\mathcal{S}$ ，令 $E^*(S)$ 表示固定原始节点序 S 后，所有合法地址、spill 选择和 spill 放置中的最小 P2 extra。给定 $\epsilon \geq 0$ ，定义近最优集合

$$\mathcal{O}_\epsilon = \{S \in \mathcal{S} : E^*(S) \leq (1 + \epsilon) \min_{S' \in \mathcal{S}} E^*(S')\}, \quad (10)$$

以及该集合内的工作集下界

$$\underline{W}_c^\epsilon = \min_{S \in \mathcal{O}_\epsilon} W_c^S. \quad (11)$$

正文定理 1 声称：若近最优 extra 子集上的工作集下界超过 cache 容量，则溢出不可避免。以下给出完整证明，并讨论该证书作为单侧条件的含义：它能确认溢出不可避免，但反向不成立。

定理 1。 若 $\underline{W}_c^\epsilon > \text{Cap}_c$ ，则每个 $S \in \mathcal{O}_\epsilon$ 都必须在 cache c 上发生容量溢出。换言之，在保持近最优 P2 extra 的候选范围内，仅靠地址摆放无法让 cache c 完全 spill-free。

更定量地，对任意 $S \in \mathcal{O}_\epsilon$ 及任意尊重 S 的合法 P2 实现 P ，

$$\text{extra}(P) \geq \max_t (W_c^S(t) - \text{Cap}_c)_+. \quad (12)$$

证明。 在任一位置 t ，将逻辑 live 体积分解为已驻留体积 $R_c^P(t)$ 与已 spill、暂不驻留体积 $A_c^P(t)$ ：

$$W_c^S(t) = R_c^P(t) + A_c^P(t). \quad (13)$$

容量约束给出 $R_c^P(t) \leq \text{Cap}_c$ ，故

$$A_c^P(t) \geq (W_c^S(t) - \text{Cap}_c)_+. \quad (14)$$

任一暂不驻留但仍 live 的缓冲，在该位置之前至少被 spill 过一次；一次 spill 的 extra 至少为该缓冲大小。因此，总 extra 不小于任一时刻的缺席 live 体积。对 t 取最大即得上式。若 $\underline{W}_c^\epsilon > \text{Cap}_c$ ，则 $\mathcal{O}_c$ 中每个序都存在某个位置使 $W_c^S(t) > \text{Cap}_c$ ，所以不可能在该 cache 上完全避免 spill。

证书的单侧性与适用范围。 该定理给出的是一个单侧证书 (one-sided certificate)：它只能确认溢出不可避免，反向不成立。当 $\underline{W}_c^\epsilon \leq \text{Cap}_c$ 时，并不能推出 spill-free；碎片、地址复用依赖、被 pin 住的驻留段和重载位置仍可能导致 spill。近最优集合限制也不可省略：若在所有合法序上无约束地最小化峰值，某些 P2 extra 很差的低峰值序会使实例看起来“不受容量限制”。例如 Matmul_Case1/L1 的全序最小工作集比值为 0.03125，但在 10% 近最优 extra 集合内的比值为 8.5，后者才反映 P2 相关区域的容量压力。正文 §5.3 的 CP-SAT oracle 验证表明，该下界在全部小图上成立。

2.3 Belady 风格驱逐的稳定条件

正文命题 1 断言，在距离主导条件下不同 Belady 风格驱逐规则会选出相同的 victim 集合。以下给出形式化条件和证明。

固定某个调度序，在一次 cache 溢出决策 k 中，令 R_k 为可驱逐驻留缓冲集合， Δ_k 为必须释放的体积， $d_k(b)$ 为缓冲 b 到下一次使用的距离。考虑一族距离主导的 Belady 风格规则：

$$\text{score}_r(b) = d_k(b)g_r(b), \quad (15)$$

其中 $g_r(b) > 0$ 是规则相关的有界扰动，例如 clean/dirty 代价权重。假设所有规则和候选满足 $g_r(b) \in [g_{\min}, g_{\max}]$ ，并记 $\rho = g_{\max}/g_{\min}$ 。

命题 1。 若存在集合 $F_k \subseteq R_k$ ，满足：(i) $\sum_{b \in F_k} s_b \geq \Delta_k$ ；(ii) F_k 的任意真子集都不足以释放 Δ_k ；(iii) 任意 $x \in F_k$ 与 $y \in R_k \setminus F_k$ 都满足

$$d_k(x) > \rho d_k(y), \quad (16)$$

则上述规则都会优先选择同一个 victim 集合 F_k ，只可能在 F_k 内部有 tie 顺序差异。若该条件在每次溢出决策均成立，则这些规则得到相同的驻留演化和相同的 P2 extra。

证明。 对任意规则 r 、 $x \in F_k$ 、 $y \notin F_k$ ，

$$\frac{\text{score}_r(x)}{\text{score}_r(y)} = \frac{d_k(x)g_r(x)}{d_k(y)g_r(y)} \geq \frac{d_k(x)g_{\min}}{d_k(y)g_{\max}} > 1. \quad (17)$$

因此每个 F_k 中的元素都排在集合外元素之前。贪心选择最高分元素直到释放足够体积时，由充分性和极小性可知它恰好选中 F_k 。

命题的直觉与适用条件。 本文采用的代价加权距离分数为 $d_k(b)s_b/e_b$ 。由于 $e_b = s_b$ 或 $2s_b$ ，clean/dirty 权重最多造成 $2\times$ 扰动。因此，当远期候选的下次使用距离超过近端候选两倍以上时，代价权重不会改变 victim 集合。若候选在大小和 clean/dirty 类别上近似同质，不同 Belady 风格规则也会退化为相同排序。这个命题解释了固定序上 victim rule 往往不敏感，但它不是无条件定理；若短期 clean 缓冲和长期 dirty 缓冲反复交替，规则差异可以累积。正文 §5.3 的受控实验证实了这一预测：固定序时四种 victim rule 的 extra 差异 $\text{CV} < 4\%$ ，而不同序之间的 extra 摆动可达 $10\times$ 以上。

2.4 溢出面积代理的条件性

令未归一化溢出面积为

$$A(S) = \sum_t \sum_c (W_c^S(t) - \text{Cap}_c)_+, \quad (18)$$

归一化代理为

$$\Phi(S) = \sum_t \sum_c \frac{(W_c^S(t) - \text{Cap}_c)_+}{\text{Cap}_c}. \quad (19)$$

二者都度量“超容量的 memory-time 面积”。P2 extra 则是事件代价：同一次 spill 可让缓冲缺席多个调度位置。因此， $A(S)$ 或 $\Phi(S)$ 与最优 extra 之间不存在无条件常数近似。长寿命小溢出会使面积随时间增长而 extra 只支付一次；反过来，一个只超出 1 单位容量的大缓冲也可能迫使换出整个大块。

正文定理 2 在有界缺席时长假设下建立溢出面积 $A(S)$ 与最优 extra 的双边常数关系。以下给出完整证明与反例构造。

定理 2。 固定序 S 。若存在合法 spill 方案 P ，其每个 spill episode 的缺席长度 ℓ_j 满足 $\ell \leq \ell_j \leq L$ ，其移除 memory-time 面积

$$R(P) = \sum_j s_j \ell_j \quad (20)$$

满足 $R(P) \leq \lambda A(S)$ ，并且每单位 spill 代价 κ_j 满足 $\kappa_{\min} \leq \kappa_j \leq \kappa_{\max}$ ，则固定序最优 extra 满足

$$\frac{\kappa_{\min}}{L} A(S) \leq E^*(S) \leq \frac{\kappa_{\max} \lambda}{\ell} A(S). \quad (21)$$

在本文评测模型中， $\kappa_{\min} = 1$ 、 $\kappa_{\max} = 2$ 。

证明要点。 任一合法实现必须在每个位置至少移除 $(W_c^S(t) - \text{Cap}_c)_+$ 的体积，故 $A(S) \leq R(P)$ 。又因 $\ell_j \leq L$ ，有 $R(P) \leq L \sum_j s_j$ ；每单位 extra 至少为 $\kappa_{\min}$ ，得到下界。上界使用假设中的可行方案：由 $\ell_j \geq \ell$ 得 $\sum_j s_j \leq R(P)/\ell \leq \lambda A(S)/\ell$ ，再乘以每单位最高代价 $\kappa_{\max}$ 。

从 A 到 Φ 。 由于各 cache 容量固定且最小容量为 256、最大容量为 4096，

$$256 \Phi(S) \leq A(S) \leq 4096 \Phi(S). \quad (22)$$

因此以 Φ 给出直接理论保证会额外支付 $16\times$ 的容量跨度因子。本文使用 Φ 的主要理由不是它经验上显著强于逐 cache 峰值，而是它以 $O(N)$ 代价给出 lifetime-aware、跨 cache 可比较的候选选择信号。正文 §5.3 报告的经验数据印证了这一定位：在评估集合中， Φ 与 P2 extra 的全局 Spearman 相关性为 0.958，逐 cache 峰值压力为 0.955，二者基本持平；定理 2 的经验比 $E^*/A \in [0.56, 1.13]$ 落在常数因子带内。这一事实也要求正文采用克制表述： Φ 的价值在于计算效率与跨阶段一致性，而非显著优于逐 cache 峰值的预测精度。

2.5 Portfolio 选择的单调性

正文方法节 (§4.2) 指出，加入候选序不会使结果退化。以下给出形式化论证。候选 portfolio 的安全性来自一个简单事实。令 $\mathcal{P}$ 为候选序、victim rule 或预取窗口的有限集合， $J(x)$ 为真实评测键（可以是词典序键）。若输出由 $\arg \min_{x \in \mathcal{P}} J(x)$ 选择，则加入新候选 z 后

$$\min_{x \in \mathcal{P} \cup \{z\}} J(x) \leq \min_{x \in \mathcal{P}} J(x). \quad (23)$$

因此，只要最终选择使用真实 P2/P3 评测键而不是代理分数，扩大候选集合不会使已选结果退化；代理 Φ 只用于生成和筛选候选，不替代最终目标。

3 方法细节与端到端对照

正文 §4 以算法伪代码给出了候选序生成与地址分配的整体结构。本节展开三个方面的设计细节：候选序的互补设计理由 (§3.1)、端到端结果与外部参考解的对照 (§3.2–§3.5)、以及差距来源的诊断分析 (§3.6)。

3.1 候选序设计与溢出分离原则

正文将调度序作为优化溢出代价的主要自由度。这一策略要求候选序集合能够覆盖峰值窗口内 clean/dirty 构成的不同形态，而非仅追求单一指标的局部最优。以下先说明三个候选序为何互补，再讨论预取窗口与溢出选择的分离设计。

候选序。 三个候选序均为结构无关的拓扑序，共享同一内存感知表调度框架（释放优先、分配延后），仅在就绪集排序策略上形成差异。

1. 压力感知序 ($p1$) 是正文算法 1 使用的内存感知表调度序，各级就绪集内采用内存感知优先级。ALLOC 节点按后继入度/后继就绪度排序：入度越低说明消费者越快就绪，分配被推迟到接近使用时放行，从而将分配时机压到峰值最不敏感处。该序是 P1 的直接输出，也是 P2/P3 的一个候选。
2. 容量节流序 ($capfit-id$) 各级就绪集内沿用内存感知的释放/操作优先级，但 ALLOC 改按最小节点标识排序，并在此基序上叠加容量门控：若某个 ALLOC 会使对应 cache 超过容量，则暂缓该分配；当所有就绪分配均被阻塞、无法再推迟时，强制放行最快解锁后续工作的那个（权重按 $1/Cap_c$ 归一化）。该序的角色是直接约束峰值高度，代价是牺牲部分缓冲的连续驻留机会。
3. 标识保留序 ($id-raw$) 在释放、操作、分配三级就绪集内一律取最小标识，既无内存感知优先级也无容量门控。其作用是保留 COPY-IN clean 缓冲的原始驻留顺序，使这些 clean 缓冲更容易在计算窗口内常驻，从而在高压窗口留下可半价换出的 clean 储备。这一效果无法由前两个序可靠地产生，因为它们的排序策略都会打乱输入缓冲的原始顺序。

三个序的互补性体现在对 clean/dirty 构成的不同塑形路径：容量节流序 ($capfit-id$) 通过容量门控降低峰值总量，压力感知序 ($p1$) 通过延迟分配缩短活跃区间，标识保留序 ($id-raw$) 通过顺序保持维持 clean 储备。正文 §4.2 已指出，候选集扩展具有单调性，加入新候选只改善或持平。

地址分配与代价感知驱逐。 给定拓扑序后，沿序执行 best-fit 地址分配。容量不足时按代价加权的 Belady 分数选择驱逐对象：

$$\text{victim} = \arg \max_b d(b) \frac{s_b}{e_b} = \arg \max_b \begin{cases} d(b), & b \text{ clean}, \\ d(b)/2, & b \text{ dirty}, \end{cases} \quad (24)$$

其中 $d(b)$ 是缓冲 b 到下一次使用的距离。dirty 缓冲的有效距离折半，使引擎优先驱逐廉价的 clean 缓冲：只有当某个 dirty 缓冲的未来使用足够远时，才会优先于同等大小的 clean 缓冲被换出。正文命题 1 表明，在距离主导条件下该评分规则与其他代价加权变体选出相同的驱逐集合，因此驱逐策略的边际收益远小于调度序。

预取窗口与溢出选择的分离。 溢出集合的选择与重载放置是两个可分离的自由度。前者决定 extra 的总量，后者影响重载能否与计算重叠，进而影响 P3 时间。将二者耦合会使时间优化无约束地污染流量目标。方法的设计选择是将二者拆开。预取窗口 H 仅控制 SPILL-IN 的前移：若某个已换出缓冲将在 $\leq H$ 步内被使用，且 cache 有空闲，则提前重载，但不为预取触发额外驱逐。P2 使用紧网格 $H \in \{0, 5, 40\}$ 以最小化流量；P3 使用宽网格 $H \in \{0, 5, 40, 80, 120\}$ 以最小化时间。

选优机制。 对 3 个候选序与预取窗口的笛卡尔积逐一执行完整模拟：P2 按 $(\text{extra}, \#\text{spills}, T)$ 词典键选优，P3 按 $(T, \text{extra}, \#\text{spills})$ 。最终输出为候选集上的最小项。

Algorithm 1 P1/P2/P3 的统一求解框架

Require: DAG $G = (V, E)$ ，问题类型 $p \in \{1, 2, 3\}$

Ensure: 调度；P2/P3 同时输出地址分配 mem 与 spill 列表 spill

```

1: if  $p = 1$  then
2:   return MemoryAwareOrder( $G$ )
3: end if
4:  $\mathcal{H} \leftarrow \{0, 5, 40\}$  if  $p = 2$  else  $\{0, 5, 40, 80, 120\}$ 
5:  $\text{best} \leftarrow \perp$ 
6: for  $\sigma \in \{\text{capfit-id}, \text{pl}, \text{id-raw}\}$  do
7:   for  $H \in \mathcal{H}$  do
8:      $(\pi, \text{mem}, \text{spill}) \leftarrow \text{AssignAndSpill}(G, \sigma, H)$ 
9:      $E \leftarrow \sum_{(b, \cdot) \in \text{spill}} e_b$ ;  $T \leftarrow \text{SimTime}(\pi, \text{mem}, \text{spill})$ 
10:     $k \leftarrow (E, |\text{spill}|, T)$  if  $p = 2$  else  $(T, E, |\text{spill}|)$ 
11:     $\text{best} \leftarrow \min(\text{best}, (k, \pi, \text{mem}, \text{spill}))$ 
12:   end for
13: end for
14: return  $\text{best}$ 

```

3.2 P1: 峰值驻留与目标错配

P1 评判逐 cache 峰值驻留，其结果直接回答“给定 DAG 能否在某个合法序下降低峰值”。然而 P1 最优序并不自动转化为 P2/P3 最优，这正是正文强调的目标错配现象。

端到端结果显示，本文序在全部六个实例上的 L1 峰值均低于外部参考解：Conv_Case0 从 7488 降至 6912，Conv_Case1 从 14040 降至 13786；FlashAttention 与 Matmul 实例的降幅更大，P1 峰值分别降至 256/256/128/128。但 P1 低峰值不等价于低 spill 代价。以 FlashAttention_Case1 为例，P1 序使 UB 峰值达到 33010，远超 $\text{Cap}_{\text{UB}} = 1024$ ：降低 L1 峰值的序反而在 UB 上造成了更严重的容量超限。这一 case 直接解释了为什么方法需要从 P1 单峰值指标过渡到 P2 的溢出流量指标：在多 cache 体系下，跨 cache 的峰值分布才是决定真实溢出代价的关键。

3.3 P2: 调度序塑形 clean/dirty 构成

P2 的核心目标是最小化总溢出流量 extra。与仅在固定序上替换驱逐策略不同，本文通过调度序改变高压窗口中 clean 与 dirty 缓冲的构成，使驱逐器在容量超限时能以 s_b (clean) 而非 $2s_b$ (dirty) 的代价释放空间。

相对外部参考解，P2 在六个实例中三胜三负。胜出实例揭示了塑形机制的有效性：Conv_Case1 (73240 $\rightarrow$ 72520) 中，本文序将更多 clean 缓冲保留到高压窗口；Matmul_Case0 (34944 $\rightarrow$ 34688) 同理；Matmul_Case1 的 extra 与 spill 数均与参考解持平 (460800、3600)，但时间第三键更优。三个失利实例则指向同一机制的反面：参考序在关键高压窗口中恰好保留了更合适的 clean 缓冲。例如 Conv_Case0 中参考解的 extra 为 73500，低于本文的 88044，差距并非来自驱逐规则的差异（命题 1 表明固定序上不同驱逐变体的 extra 差异很小），而是参考序使更多 clean 缓冲在溢出发生时仍然驻留，正体现了调度序对驻留构成的塑形效果。

胜与负共同印证了正文的核心论点：溢出代价的主要决定因素是序对 clean/dirty 构成的塑形，而非驱逐规则本身。

3.4 P3: 预取窗口与计算重叠

P3 以流水化执行时间为首要目标。非零预取窗口将部分 SPILL-IN 从直接使用点前移，使重载与计算或其他管线搬运重叠，降低用户节点的等待阻塞。由于预取仅在存在空闲区间时发生且不触发额外驱逐，它主要调整重载放置时机，而非重新定义 spill 集合。

端到端结果为四胜二负。本文在两个 FlashAttention 与两个 Matmul 实例上取得更短时间：FlashAttention_Case1 从 193059 降至 180364，Matmul_Case0 从 194331 降至 186820。两个 Conv 实例时间落后，其中 Conv_Case1 尤其具有诊断价值：本文 extra 更低 ($73578 < 86690$)，但参考解时间更短 ($1073322 < 1111778$)。这一反例说明更低的流量不自动转化为更短的时间：重载与计算的重叠程度是另一个独立的维度。当参考程序的重载恰好被安排在计算管线的空闲间隙中时，即使总流量更高，阻塞时间仍可更短。这一观察进一步支持了方法将溢出选择与重载放置分离的设计决策。

3.5 端到端参考对照

表 S3 汇总六个公开实例在 P1/P2/P3 上的端到端结果，合计 13/18 胜。这里的比较口径与正文主实验不同：正文 §5.2 使用“同引擎换序”口径，所有方法共享同一地址分配/溢出引擎、仅替换调度序，以隔离序对 clean/dirty 构成的塑形效果。端到端对照则使用各自完整的调度、地址分配和 spill 列表，反映两个完整系统的最终输出质量。两类口径回答不同的问题，端到端数据作为系统性能的综合参照。

表 S3: 本文方法与外部参考解 baseline01 的端到端对照（每格为“本文/参考”）。

Case	P	max _{L1}	spills	extra	T	结果
Conv_Case0	P1	6912/7488	0/0	0/0	394430/359570	胜
	P2	6912/7488	339/141	88044/73500	615694/535312	负
	P3	6912/7488	342/141	92652/73500	553280/535312	负
Conv_Case1	P1	13786/14040	0/0	0/0	650044/573395	胜
	P2	14040/14040	1963/1828	72520/73240	1159169/1150333	胜
	P3	14040/14040	1951/1290	73578/86690	1111778/1073322	负
FA_Case0	P1	256/3328	0/0	0/0	31063/31429	胜
	P2	3328/3328	31/41	3904/3692	51700/53374	负
	P3	4096/3328	58/24	5832/4128	46167/46761	胜
FA_Case1	P1	256/5376	0/0	0/0	112458/113445	胜
	P2	5376/5376	229/146	32920/32840	214361/193059	负
	P3	5376/5376	229/146	34432/32840	180364/193059	胜
Matmul_Case0	P1	128/9216	0/0	0/0	82308/82773	胜
	P2	9216/9216	271/273	34688/34944	192380/194331	胜
	P3	9216/9216	273/273	34944/34944	186820/194331	胜
Matmul_Case1	P1	128/34816	0/0	0/0	629076/583525	胜
	P2	34816/34816	3600/3600	460800/460800	1783145/1800218	胜
	P3	34816/34816	3601/3600	460928/460800	1771132/1800218	胜

3.6 差距来源分析

五个失利项的核心发现一致：参考解在这些实例中的优势不削弱正文论点，反而从反面印证了同一机制，即差距主要来自调度序对 clean/dirty 驻留构成的塑形，而非固定序上的驱逐策略差异。

两类诊断交叉验证了这一判断。第一，将参考解的序输入本文地址分配/溢出引擎后，若干失利项的

流量可被接近复现，说明差距来自高压窗口中可供驱逐的缓冲构成，而非引擎实现的差异。第二，在固定序上替换四类 Belady 风格 victim policy，P2 extra 的差异通常很小（正文 §5.3 报告 $CV \leq 4\%$ ）；相反，不同合法序之间的 extra 可相差一个数量级。

本文保留这些失利项而不通过算子族特化规则去拟合，是因为此类拟合会破坏方法的结构无关性：算法只使用 DAG 依赖、缓冲大小、cache 类型、clean/dirty 属性和容量信息，不包含按算子类别定制的排序规则。一旦引入特化，结论将从”调度序塑形 clean/dirty 构成降低溢出代价”退化为”针对特定算子结构做专门排序”，削弱方法的一般性。

3.7 运行效率

表 S4 给出完整的 3 次重复统计（中位数、最小值、最大值），补充正文表 3 的中位数数据。

表 S4: Wall-clock 运行时间（秒，3 次重复）。

实例	P	中位数	最小值	最大值	节点数
Conv_Case0	P1	0.009	0.009	0.010	2 580
	P2	0.391	0.390	0.392	
	P3	0.683	0.677	0.706	
Conv_Case1	P1	0.189	0.186	0.190	36 086
	P2	57.97	57.46	58.62	
	P3	73.14	73.10	73.28	
FA_Case0	P1	0.006	0.006	0.007	1 716
	P2	0.220	0.218	0.222	
	P3	0.328	0.326	0.328	
FA_Case1	P1	0.027	0.026	0.030	6 952
	P2	2.723	2.723	2.757	
	P3	3.356	3.316	3.367	
Matmul_Case0	P1	0.014	0.014	0.016	4 160
	P2	0.701	0.695	0.703	
	P3	1.137	1.136	1.170	
Matmul_Case1	P1	0.126	0.111	0.126	30 976
	P2	19.40	19.33	19.46	
	P3	32.33	32.19	32.34	

运行时间的主导因素。 P1 仅生成单个调度序，所有实例均低于 0.2 秒。P2/P3 的运行时间主要受两个因素支配：(1) 候选序 $\times$ 预取窗口的网格大小（P2 评估 $3 \times 3 = 9$ 个组合，P3 评估 $3 \times 5 = 15$ 个组合）；(2) 地址分配中 Belady 驱逐选择的二次最坏开销。Conv_Case1 和 Matmul_Case1 的缓冲数分别为 12 013 和 8 960，驱逐扫描构成运行时间的主要部分。三次重复的最小/最大值紧密包围中位数（变异系数 $< 2\%$ ），说明计时结果稳定。

时间复杂度。 设 $N = |V|$ 为节点数， $E = |E|$ 为边数， B 为缓冲数， $K = 3$ 为候选序数， W 为预取窗口网格大小。各阶段复杂度如下：

- **候选序生成：** $O(K(N \log N + E))$ ，基于堆维护的就绪集拓扑排序。
- **地址分配与溢出插入：** $O(N + E + B^2)$ 每个（序，窗口）组合。沿序扫描线性，Belady 驱逐的最坏开销为 B 次分配各扫描 $O(B)$ 个驻留缓冲。
- **流水模拟：** $O(N + E)$ 每个组合。

- **Portfolio 选优**: $O(KW)$ 个组合的枚举。

综合为 $O(KW(N \log N + E + B^2))$ 。由于 K 、 W 为小常数，渐近主项为 $O(N \log N + E + B^2)$ 。空间复杂度为 $O(N + E + B)$ 。

4 附加实验证据

正文实验节只保留了最能支撑主线的图表。本节补充更细粒度的证据，并按论证角色组织：同引擎换序对照与敏感性分析用于隔离”调度序”这一自由度；合成分布用于说明方法适用域；CP-SAT oracle 用于核验理论下界与给定序最优性；代理目标、工作集和预取窗口实验用于解释算法设计；clean/dirty 消融用于隔离 $2\times$ 代价梯度。

4.1 调度序的作用

正文 §5.3 的受控对照表明固定序时驱逐规则差异不超过 4%，但不同序的 extra 可摆动一个数量级。本节用三组互补的实验量化这一发现：同引擎换序对照、驱逐策略敏感性分析、以及跨序摆动可视化。

表 S5 给出六个公开实例在 P2 视角下的受控对照。所有方法共享同一地址分配与 spill 引擎，仅替换输入拓扑序。因此，extra 的差异可归因于调度序改变了高压窗口内的驻留构成，而不是归因于不同分配器或不同 victim policy。

表 S5: 受控对照: P2 溢出流量 (同一地址/spill 引擎, 仅替换调度序)。

Case	CP-list	CP-free	pressure	G-Hsu	ours
Conv_Case0	694,506	212,956	207,932	210,844	88,044
Conv_Case1	1,695,264	73,348	1,048,524	1,053,484	72,520
FA_Case0	211,784	3,904	101,356	90,092	3,904
FA_Case1	965,944	33,152	445,876	418,228	32,920
Matmul_Case0	823,168	34,944	296,064	298,240	34,688
Matmul_Case1	6,506,240	460,800	2,556,928	2,560,640	460,800

clean/dirty 无感的内存压力调度器 (uniform pressure 与 Goodman-Hsu) 在全部真实 case 上均显著高于本文序, extra 倍数约为 $2.4\text{--}26\times$ 。关键路径导向的 CP-list 差距更大。相比之下, CP-free 是一个强 companion: 它通过优先释放节点间接缩短生命期, 在多数组合上接近本文序, 但在 Conv_Case0 等 clean 储备对代价起决定作用的实例上仍明显落后。这一结果说明, 仅降低活跃字节或提前释放并不足以解释全部收益; 高压窗口中 clean/dirty 组成的差异是必要因素。

为直接检验驱逐策略的边际效应, 图 S1 在固定调度序上替换四种 Belady 风格 victim rule, 比较各 case 的 extra 变化。结果显示多数 Matmul 与 FlashAttention 实例上四种策略完全重合, Conv 行存在可见差异但量级仍远小于跨序差异。这与命题 1 的理论预测一致: 在距离主导条件下, 驱逐策略的边际收益有限。

反过来, 图 S2 展示不同合法拓扑序在同一引擎下的 extra 摆动。替换合法拓扑序会显著改变高压窗口中可驱逐缓冲的组成, 因此带来远大于驱逐规则微调的 extra 差距。这两组实验从正反两面确认: 驱逐策略仅是局部修补, 调度序才是驱动溢出代价的主要自由度。

三组证据共同支持”调度序是主要优化自由度”的论点。表 S5 量化了同引擎换序的差距, 图 S1 表明固定序下驱逐策略的边际有限, 图 S2 给出跨序摆动的数量级差异。可见溢出代价的差距主要来自调度序对峰值窗口中 clean/dirty 驻留构成的塑形, 而非驱逐规则可单独弥补。

本文涉及两种比较口径。表 S5 为同引擎换序对照, 用于隔离科学机制; 表 S3 为端到端参考对照, 比较各自完整输出。前者支撑”调度序塑形驻留构成”的论点, 后者反映最终求解器在公开实例上的提交质量, 二者不应混用。

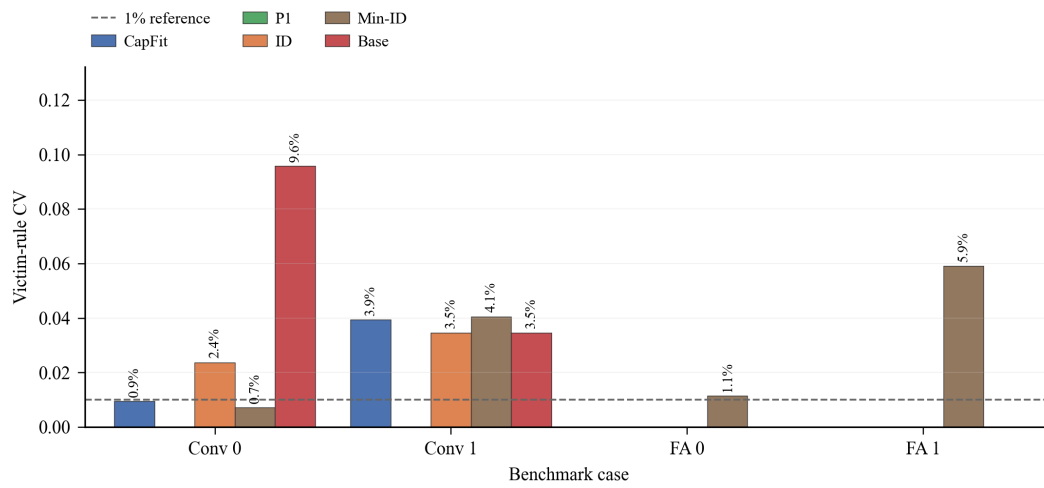


图 S1: 固定调度序下的 victim rule 敏感性。多数 Matmul 与 FlashAttention 行完全重合；Conv 行存在可见差异，但量级仍小于跨序差异。

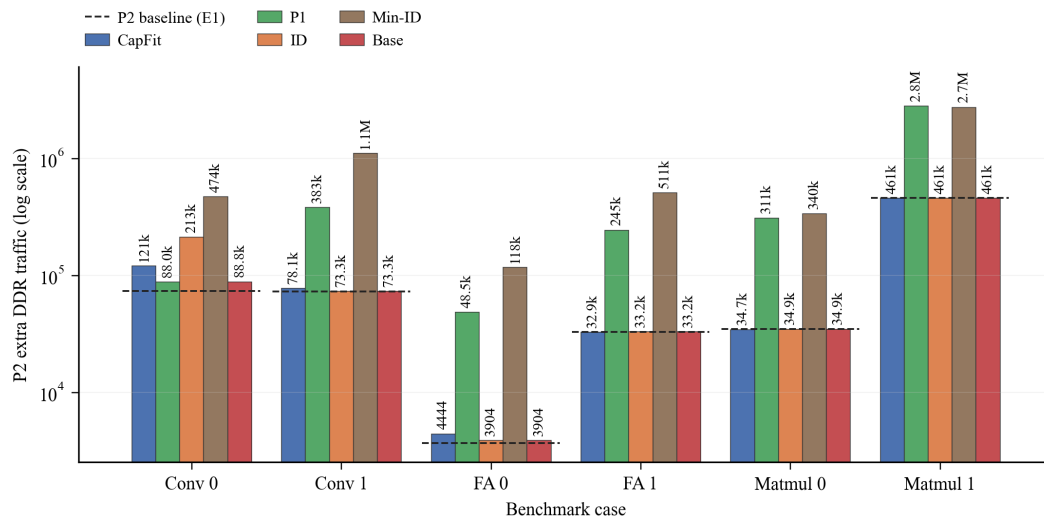


图 S2: 调度序敏感性。相比在固定序上微调驱逐规则，替换合法拓扑序会显著改变高压窗口中可驱逐缓冲的组成，因此带来更大的 extra 摆动。

4.2 合成分布与适用域

正文图 6 展示了 capacity-bound 区域的系统优势。本节给出完整的 36 个合成实例的逐实例数据与分区统计。

图 S3 与表 S6 报告 36 个合成实例上的分布结果。我们将实例分为 capacity-bound 与 order-reachable 两类：前者在容量压力下不可避免需要 spill，后者存在可通过好序直接避免 spill 的空间。该划分对应正文溢出不可避免性证书的使用方式：它不是完整可行性判定，而是识别“必须优化 spill 代价构成”的区域。

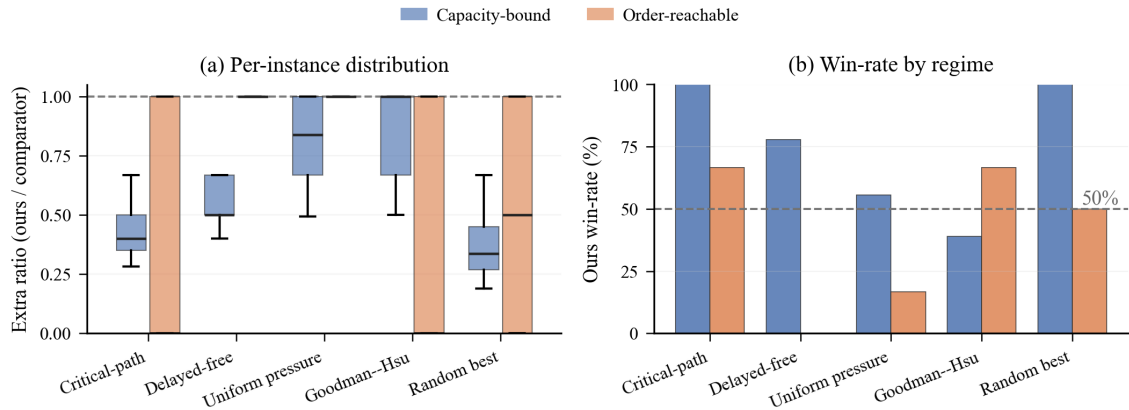


图 S3: 合成实例分布。(a) 逐实例 extra 比值；(b) 按 capacity-bound 与 order-reachable 分区统计胜率。优势主要集中在容量受限区域。

表 S6: 合成 suite 分布统计。中位倍数为 ours/comparator，数值越低表示本文序越优。

Comparator	Regime	中位 ours/comp	胜率
CP-list	all	0.39	83.3%
	capacity_bound	0.40	100%
	order_reachable	0.00	66.7%
CP-free	all	1.00	38.9%
	capacity_bound	0.50	77.8%
	order_reachable	1.00	0%
Pressure	all	1.00	36.1%
	capacity_bound	0.84	55.6%
	order_reachable	1.00	16.7%
G-Hsu	all	0.67	52.8%
	capacity_bound	1.00	38.9%
	order_reachable	0.00	66.7%
Random-best	all	0.34	75.0%
	capacity_bound	0.34	100%
	order_reachable	0.50	50.0%

结果与理论定位一致。capacity-bound 子群中，本文相对 CP-list 与 random-best 全胜，相对 CP-free 的中位优势为 2 $\times$ 。order-reachable 子群中，好序可以直接消除 spill，clean/dirty 塑形自然不再提供系统优势；本文相对 CP-free 的胜率降为 0%。因此，方法的目标不是支配所有 DAG，而是在 spill 不可避免时降低每单位 spill 的真实代价。

4.3 CP-SAT oracle

正文 §5.3 报告了 CP-SAT oracle 的汇总结论。本节给出逐实例数据，包括容量约束比、理论下界和固定序最优 extra。小图 oracle 用于回答两个问题：给定本文选择的序，地址/spill 引擎是否达到 capacity-only 最优；理论下界与面积代理是否在精确解上保持一致。

表 S7 给出 8 个 capacity-bound 小图的结果。 $\text{ratio}_{W^*} = W^*/\text{Cap} > 1$ 表示该实例处于容量受限状态； lb_{T1} 是正文溢出不可避免性证书给出的下界； E^* 是固定序下的最优 extra。

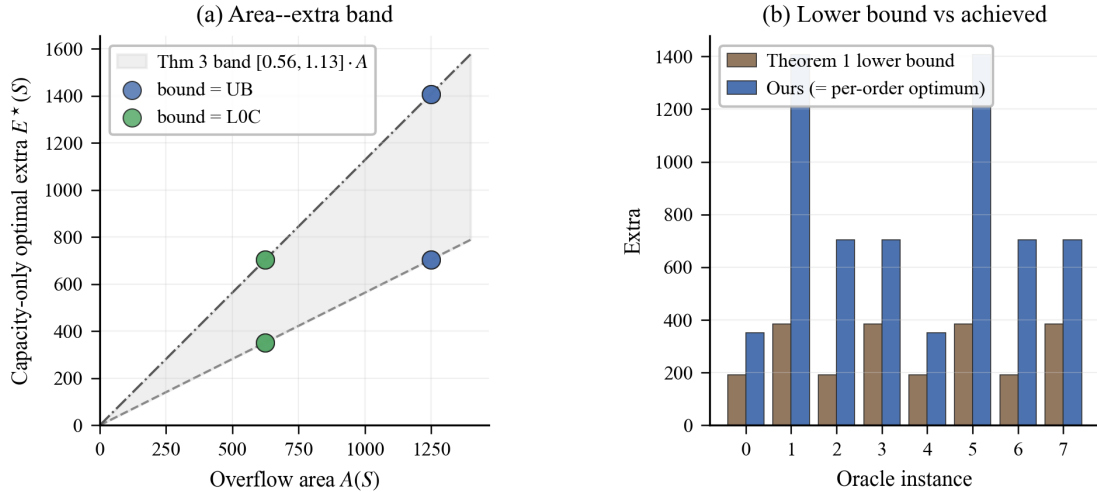


图 S4: CP-SAT oracle 验证。(a) 溢出面积 $A(S)$ 与固定序最优 extra $E^*(S)$ 的经验常数带；(b) 溢出不可避免性下界与固定序最优 extra 的对照。

表 S7: Oracle 逐实例数据。所有实例均满足 $\text{lb}_{T1} \leq E^*$ ，且本文引擎在固定序上达到 E^* 。

inst	bound	Cap	ratio_{W^*}	lb_{T1}	E^*	E^*/A
oracle_000	L0C	512	1.375	192	352	0.564
oracle_001	UB	1024	1.375	384	1408	1.128
oracle_002	L0C	512	1.375	192	704	1.128
oracle_003	UB	1024	1.375	384	704	0.564
oracle_004	L0C	512	1.375	192	352	0.564
oracle_005	UB	1024	1.375	384	1408	1.128
oracle_006	L0C	512	1.375	192	704	1.128
oracle_007	UB	1024	1.375	384	704	0.564

所有小图上，本文引擎相对固定序最优的 gap 为 1.0，说明在该组 oracle 中瓶颈不在分配/驱逐实现，而在序选择。溢出不可避免性下界均成立，且 $E^*/A \in [0.56, 1.13]$ ，落在正文所述常数因子解释范围内。跨序联合最优模型需要将排列变量与驻留变量耦合，CP-SAT 求解频繁超时；因此本文不声称相对全局最优的 gap，只使用固定序 oracle 验证局部最优性和理论界。

4.4 代理目标与诊断工具

正文 §5.3 以汇总形式报告了溢出面积代理、工作集证书和预取窗口的实验结论。本节给出这些诊断工具的逐实例数据和可视化，并补充 P1 与 P2/P3 错配的量化分析。

溢出面积代理。 图 S5 比较溢出面积代理 Φ 与逐 cache 峰值压力对 P2 extra 的解释能力。两者的 Spearman 相关性非常接近 (0.958 与 0.955)。因此，本文不把 Φ 表述为经验上显著更强的预测器；它的价值在于计算代价低、可在候选序筛选中稳定使用，并且与正文的面积-extra 近似解释一致。

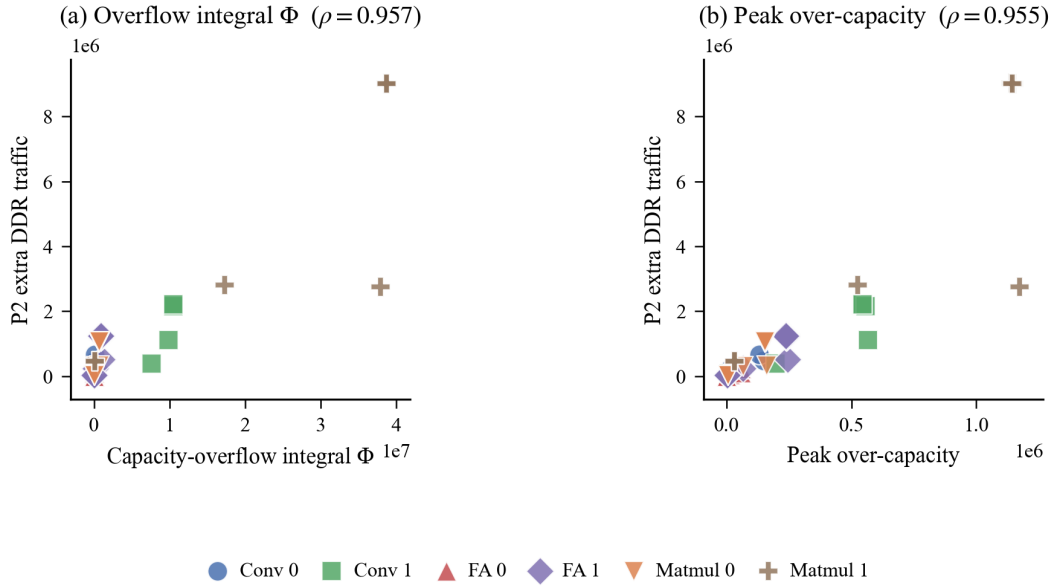


图 S5: 溢出面积代理 Φ 与峰值压力的对照。(a) Φ 对 P2 extra 的解释能力; (b) 逐 cache 峰值超容量对 P2 extra 的解释能力。二者相关性接近, Φ 的主要作用是作为便宜且跨阶段一致的选择信号。

工作集证书。 图 S6 报告近最优 extra 子集上的工作集/容量比。该限定很重要: 若跨所有序无约束地取最小峰值, 一个 P2 代价很高的低峰值序可能会错误地削弱容量受限诊断。Matmul_Case1/L1 的近最优口径为 8.5, 说明该实例处于强 capacity-bound 区域, 剩余优化空间主要来自 spill 代价构成, 而非简单消除 spill。

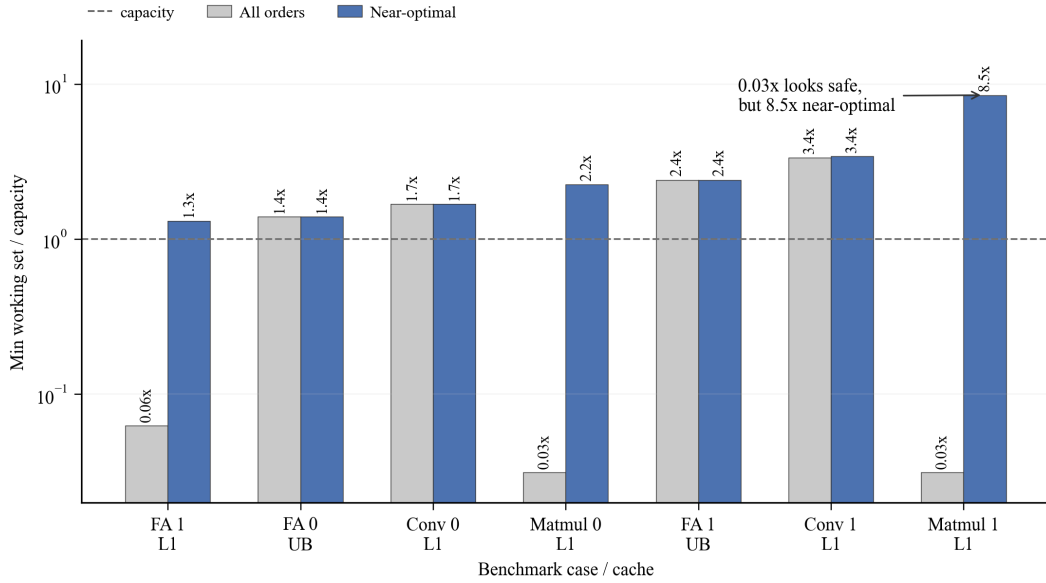


图 S6: 近最优 extra 子集上的工作集/容量比。红色项显示 Matmul_Case1/L1 在近最优口径下远超容量。

P1 与 P2/P3 的目标错配。 P1 以压低逐 cache 峰值为目标, 但正文 §4.1 指出低峰值不等价于低 spill 代价。图 S7 量化了这一错配: 单独压低 P1 目标不保证容量受限下的可行性或低 spill 代价。在一些实例上, P1 最优序的 P2 extra 反而高于本文方法选出的序, 原因在于 P1 不区分 clean/dirty 储备: 一个以低峰值为唯一目标的序可能在高压窗口中堆积大量 dirty 缓冲, 导致实际驱逐代价上升。溢出面积代理 Φ 的选择信号更直接地反映多 cache 溢出压力, 从而避免了这种错配。

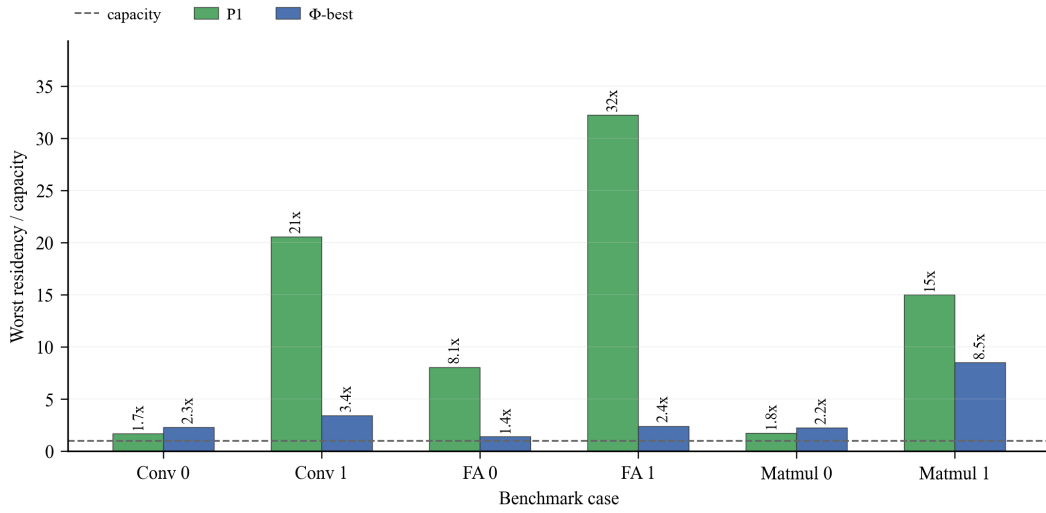


图 S7: P1 与 P2/P3 的错配。单独压低 P1 目标不保证容量受限下的可行性或低 spill 代价; Φ 选择更直接反映多 cache 溢出压力。

预取窗口解耦。 图 S8 展示预取窗口 H 对 P3 time 与 extra 的影响。非零窗口可能明显降低时间, 但 extra 往往保持不变或小幅上升。这说明 spill 集合选择与重载放置是两个相关但可分离的自由度: P2 需要保守控制流量, P3 则可以用更宽窗口换取流水重叠。

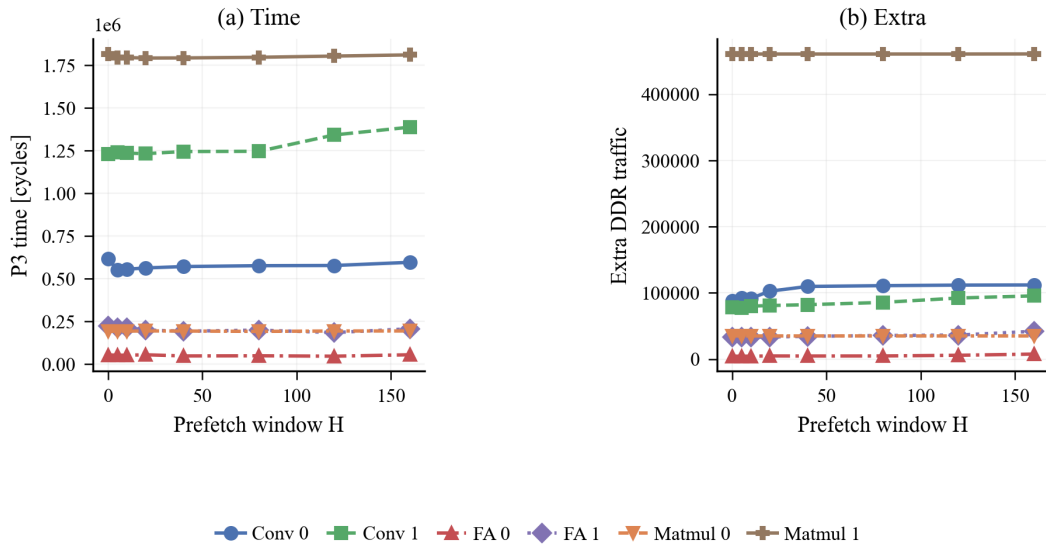


图 S8: 预取窗口 H 对时间与额外搬运量的影响。(a) P3 time 随 H 的变化; (b) extra 随 H 的变化。预取主要移动 SPILL-IN 的放置时机, 而不是重新定义 spill 集合。

4.5 Clean/dirty 受控消融

正文 §5.3 的消融实验报告了 clean 储备 extra 为 1,536、dirty 为 3,072 的 2× 代价差异。本节给出完整的实验设计和结果, 以排除公开样例的 motif 偶然性。

图 S9 使用合成 GEMM 核隔离 clean/dirty 代价梯度。其中 (a) 固定 DAG 结构、峰值和 spill 数, 仅改变高压窗口中可供驱逐的储备类型, clean 储备的 extra 为 1536, dirty 储备为 3072, 恰为 2×; (b) 在同一 clean 实例上扫描多种序, 可见 peak residency 中 clean 占比越高, extra 越低。该实验排除了公开样例 motif 的影响, 直接验证正文关于 clean/dirty 2× 代价非对称的机制论点。

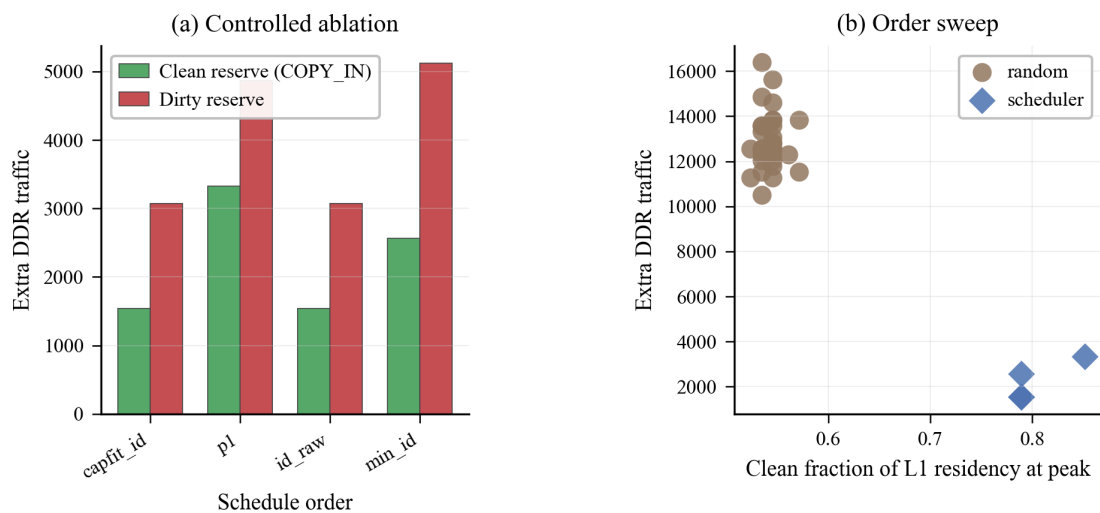


图 S9: 合成核上的 clean/dirty 机制验证。(a) 受控消融隔离 $2\times$ 代价梯度; (b) extra 随峰值处 clean 占比上升而下降。